

Формат JSON

Что такое JSON

JSON (JavaScript Object Notation — «запись объектов JavaScript») — текстовый формат для хранения и передачи структурированных данных. В нём данные отправляются на сервер и приходят с сервера, сохраняются в локальное хранилище (localStorage) и т.д.

Важно: JSON выглядит как объект, но это **не объект, а строка** — текстовое представление данных.

```
{
  "name": "Josh",
  "weight": 175,
  "isHappy": true,
  "cars": ["Chevy", "Honda"],
  "favoriteBook": {
    "title": "The Last Kingdom",
    "rating": 8.38
  }
}
```

Правила синтаксиса (отличия от объекта JS)

Что	Правило в JSON
Ключи	Всегда строки в двойных кавычках: "name"
Строковые значения	Только в двойных кавычках (одинарные нельзя)
Числа, true / false	Как в JS: целые, дробные, отрицательные
Последняя запятая	После последнего свойства запятой нет
null / undefined	null можно, undefined — нельзя
Функции (методы)	Нельзя — JSON хранит только данные

JSON.stringify — из JS в JSON

Встроенный класс JSON умеет переводить данные в обе стороны. Метод JSON.stringify(value) превращает значение (объект, массив, число, boolean, null) в JSON-строку.

```
const dog = {
  name: "Mango",
  age: 3,
  isGoodBoy: true,
};

const json = JSON.stringify(dog);
console.log(json); // '{"name":"Mango","age":3,"isGoodBoy":true}'
```

Результат — обычная строка, которую можно сохранить в хранилище, базу данных или отправить на сервер.

Что происходит с функциями

- **Методы объекта игнорируются** — в JSON они просто не попадут.
- **Функция сама по себе** превращается в undefined.

```
const dog = {
  name: "Mango",
  bark() { console.log("Woof!"); },
};
JSON.stringify(dog);           // '{"name":"Mango"}' — метода нет
JSON.stringify(() => {});      // undefined
```

JSON.parse — из JSON в JS

Парсинг (parse — «разбор») — обратная операция: `JSON.parse(value)` принимает JSON-строку и возвращает обычные JS-данные.

```
console.log(JSON.parse("5"));    // 5
console.log(JSON.parse("false")); // false

const json = '{"name":"Mango","age":3,"isGoodBoy":true}';
const dog = JSON.parse(json);
console.log(dog.name); // "Mango" — работаем как с обычным объектом
```

Метод	Направление	Результат
<code>JSON.stringify(obj)</code>	JS → JSON	строка
<code>JSON.parse(str)</code>	JSON → JS	объект / массив / значение

Обработка ошибок: try...catch

Если передать в `JSON.parse` невалидный JSON, возникнет ошибка, и **весь скрипт остановится** — код ниже не выполнится.

```
JSON.parse("Well, this is awkward"); // Ошибка: обычный текст — не JSON
JSON.parse('{username: "Mango"}');   // Ошибка: ключ без двойных кавычек
console.log("Этот лог не появится");
```

Чтобы «поймать» ошибку и не уронить скрипт, используют конструкцию `try...catch`:

```
try {
  const data = JSON.parse("Well, this is awkward");
} catch (error) {
  console.log(error.name);    // "SyntaxError"
  console.log(error.message); // Unexpected token W in JSON at position 0
}

console.log("Скрипт продолжает работать");
```

Как это работает

1. Сначала выполняется код в блоке `try`.
2. Нет ошибок — блок `catch` пропускается.
3. Ошибка в `try` — его выполнение прерывается, управление переходит в `catch`. Код после конструкции продолжает работать.

Объект ошибки `error`

- `name` — тип ошибки (для парсинга — `SyntaxError`).
- `message` — описание, что пошло не так.

- `stack` — стек вызовов функций в момент ошибки (для отладки).

Две фазы выполнения кода

Фаза	Что происходит	Какие ошибки
Оценка (compile time / evaluation time)	Интерпретатор читает код и проверяет, можно ли его вообще запустить. Код ещё не выполняется.	Синтаксические — ошибки парсинга (опечатки, неверные конструкции)
Выполнение (runtime)	Код реально выполняется: вызываются функции, вычисляются выражения, ищутся переменные.	Ошибки времени выполнения: нет переменной или свойства, неверный тип и т.п.

```
console.log("Это сообщение не появится");
cos value = 5; // опечатка вместо const — синтаксическая ошибка
```

Здесь ошибка находится ещё на фазе оценки, поэтому фаза выполнения даже не начнётся — не выполнится ни одна строка, включая первый `console.log`.

Запомнить: `try...catch` ловит только ошибки **времени выполнения** (runtime errors). Код внутри должен быть синтаксически правильным — иначе скрипт не запустится вовсе. Ошибка `JSON.parse` — это runtime-ошибка (строка разбирается во время работы), поэтому её `try...catch` поймать может.

Вебхранилище

Cookie и Web Storage

Cookie («куки») — небольшие текстовые данные в виде пар «ключ–значение», которые хранятся в браузере и **автоматически отправляются на сервер с каждым HTTP-запросом** (HTTP — HyperText Transfer Protocol, протокол передачи данных в вебе). У cookie есть срок жизни, после которого браузер их удаляет. С cookie обычно работают на сервере через библиотеки, фронтенд трогает их редко.

Для хранения данных на стороне клиента в HTML5 появился **Web Storage API** (API — Application Programming Interface, программный интерфейс): **localStorage** и **sessionStorage**. Они доступны только JavaScript-коду в браузере.

	Cookie	localStorage	sessionStorage
Объём	~4 КБ	5–10 МБ	5–10 МБ
Срок жизни	до даты истечения	бессрочно, пока не удалят вручную или через JS	пока открыта вкладка
Отправляется на сервер	да, с каждым запросом	нет	нет
Общее для вкладок	да	да, для всех вкладок сайта	нет, у каждой вкладки своё

Что хранят: состояние интерфейса — тема (светлая/тёмная), товары в корзине, открыт ли сайдбар, способ оплаты, адрес доставки, черновик формы.

Нельзя хранить пароли, номера карт и другие конфиденциальные данные: вредоносный скрипт на странице легко их прочтает.

Где посмотреть в браузере

DevTools (инструменты разработчика) → вкладка **Application** → раздел **Storage** → **Local Storage** / **Session Storage**. Там видно все записи сайта, их можно редактировать и удалять.

localStorage

- У каждого сайта (домена) своё хранилище — данные разных сайтов не пересекаются.
- Данные не удаляются при закрытии вкладки, браузера или выключении компьютера.
- Доступ через `window.localStorage` или просто `localStorage`.
- Ключ — всегда строка. Значение — **тоже только строка**.

```
console.log(localStorage); // Storage {length: 0}
```

Методы

Метод	Что делает
<code>setItem(key, value)</code>	Записывает значение по ключу. Если ключ уже есть — перезаписывает
<code>getItem(key)</code>	Возвращает значение (строку) или <code>null</code> , если ключа нет
<code>removeItem(key)</code>	Удаляет одну запись, ничего не возвращает
<code>clear()</code>	Полностью очищает хранилище

```
localStorage.setItem("ui-theme", "light");
console.log(localStorage.getItem("ui-theme")); // "light"
console.log(localStorage.getItem("no-such-key")); // null
```

```
localStorage.removeItem("ui-theme");
console.log(localStorage.getItem("ui-theme")); // null
```

```
localStorage.clear(); // Storage {length: 0}
```

Осторожно с `clear()` : он удалит и записи, созданные другими разработчиками проекта. Лучше удалять только ненужные ключи через `removeItem`.

Сохранение объектов и массивов

Поскольку хранилище принимает только строки, сложные данные нужно:

- **при записи** — превратить в строку через `JSON.stringify()` ;
- **при чтении** — разобрать обратно через `JSON.parse()` .

Строки-примитивы парсить не нужно. Функции и методы в хранилище не попадают.

```
const settings = {
  theme: "dark",
  isAuthenticated: true,
  options: [1, 2, 3],
};
localStorage.setItem("settings", JSON.stringify(settings));

const savedSettings = localStorage.getItem("settings"); // строка
const parsedSettings = JSON.parse(savedSettings);      // объект
console.log(parsedSettings.theme); // "dark"
```

sessionStorage

Работает **так же**, как `localStorage`, с теми же методами — только через объект `sessionStorage`. Разница — в сроке жизни: данные живут, пока открыта вкладка. Каждая новая вкладка или окно получает своё, пустое хранилище.

```
sessionStorage.setItem("user-id", "123");
sessionStorage.setItem(
  "tickets",
  JSON.stringify({ from: "Lviv", to: "Kyiv", quantity: 2 })
);

const tickets = JSON.parse(sessionStorage.getItem("tickets"));
console.log(tickets); // { from: "Lviv", to: "Kyiv", quantity: 2 }

sessionStorage.removeItem("tickets");
sessionStorage.clear();
```

Когда использовать: многошаговые формы (бронирование билетов, отелей) — чтобы сохранить данные предыдущих шагов, пока пользователь не закрыл вкладку; любые временные данные, которые не нужны надолго.

Кейс: форма, которая не теряет текст

Проблема: пользователь ввёл сообщение, но перезагрузил страницу, не отправив форму, — текст пропал.

Решение в три шага:

1. На событие `input` сохранять текущее значение поля в `localStorage` (делегирование: слушаем форму, значение берём из `evt.target`).
2. При отправке формы удалять сохранённое значение через `removeItem`.
3. При загрузке страницы подставлять сохранённый текст в поле.

```
<form class="feedback-form">
  <textarea name="message"></textarea>
  <button type="submit">Send feedback</button>
</form>
```

```
const form = document.querySelector(".feedback-form");
const textarea = form.elements.message;
const localStorageKey = "goit-example-message";
```

```
// 3. При загрузке страницы восстанавливаем текст (или пустая строка)
textarea.value = localStorage.getItem(localStorageKey) ?? "";
```

```
// 1. Сохраняем при каждом вводе
form.addEventListener("input", (evt) => {
  localStorage.setItem(localStorageKey, evt.target.value);
});
```

```
// 2. При отправке очищаем хранилище и форму
form.addEventListener("submit", (evt) => {
  evt.preventDefault();
  console.log(evt.target.elements.message.value);
  localStorage.removeItem(localStorageKey);
  form.reset();
});
```

?? (оператор нулевого слияния) подставляет "", если `getItem` вернул `null` — то есть сохранённого текста ещё нет.

Инструменты веб-разработки

Зачем нужна автоматизация

Инструменты автоматизации берут на себя рутину, чтобы разработчик занимался функциональностью:

- **Сборка** — объединение и минификация (сжатие) JS и CSS, компиляция SASS/LESS в CSS, оптимизация картинок.
- **Совместимость** — генерация кода, который работает в разных браузерах.
- **Линтинг** (проверка кода на ошибки и стиль) и автотесты.
- **Управление пакетами** — установка, обновление, удаление библиотек (npm).
- **Продакшн-сборка** — готовый к публикации (деплой) код.

Vite — сборщик проекта

Vite (с французского «быстрый», читается «вит») — инструмент для сборки и разработки веб-проектов на JavaScript.

- **Скорость** — изменения в коде видны в браузере почти мгновенно.
- **HMR** (Hot Module Replacement — «горячая замена модулей») — обновляет изменённый модуль без перезагрузки страницы.
- **ESM** (ECMAScript Modules — стандартные JS-модули) — модули загружаются по мере необходимости.
- Поддержка JavaScript, TypeScript, фреймворков (React, Vue, Svelte), CSS-препроцессоров (SASS, LESS) и плагинов.

Node.js и npm

Node.js — среда выполнения JavaScript вне браузера. На ней работают все современные инструменты фронтенда, поэтому её нужно установить на компьютер.

1. Скачать **LTS**-версию (Long-Term Support — версия с долгосрочной поддержкой, самая стабильная) с сайта **nodejs.org**.
2. При установке поставить галочку **Automatically install the necessary tools**.
3. После установки перезагрузить компьютер.

npm (Node Package Manager — менеджер пакетов Node.js) ставится вместе с Node.js. Проверка установки:

```
node --version  # выведет версию Node.js
npm --version   # выведет версию npm
```

Первичная настройка — имя и почта автора проектов (почта — та же, что на GitHub):

```
npm config set init.author.name "YOUR_NAME"
npm config set init.author.email "YOUR_EMAIL"
```

Создание проекта на Vite

В терминале перейти в папку со всеми проектами командой `cd` (Change Directory — «сменить папку»). На Windows лучше держать эту папку в корне диска (C: или D:).

```
cd C:/path/to/projects  # Windows
cd ~/projects            # macOS / Linux, папка в домашнем каталоге

npm create vite@latest
```

Дальше мастер задаёт вопросы:

1. Установить `create-vite` ? → `y` + Enter.

- Имя проекта (папки) → например, `my-vite-project`.
- Фреймворк → **Vanilla** (чистый JS, без фреймворков).
- Язык → **JavaScript**.

Получаем каркас: `index.html`, `main.js`, `style.css`, `package.json`, `.gitignore`. Переходим в папку проекта и открываем её в VS Code.

package.json

Главный файл настроек проекта: хранит список зависимостей, служебную информацию и скрипты. Обновляется автоматически при установке/удалении пакетов, но скрипты можно дописывать вручную.

```
{
  "name": "my-vite-project",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "vite": "^4.4.5"
  }
}
```

Установка зависимостей

После создания проекта (и после клонирования чужого) в папке проекта выполняем:

```
npm install # или коротко: npm i
```

Файл / папка	Что это
node_modules	Все скачанные пакеты. Никогда не загружается на GitHub — у каждого разработчика своя; исключена через <code>.gitignore</code>
package-lock.json	Точный «снимок» версий всех зависимостей — чтобы у всей команды стояли одинаковые версии. Обновляется npm автоматически
.gitignore	Список файлов и папок, которые Git не отслеживает

Запуск сервера разработки

Поле `scripts` — это короткие имена (псевдонимы) для команд. Имя свойства — название команды, значение — что выполнить. Запуск:

```
npm run [script-name]
```

```
npm run dev      # локальный сервер разработки → http://localhost:5173
npm run build    # продакшн-сборка
npm run preview  # просмотр готовой сборки
```

Локальный сервер работает на твоём компьютере: сайт открывается в браузере, а изменения в файлах сразу видны на экране. Процесс не завершается сам и занимает терминал.

Остановить сервер: **Ctrl + C** (Windows) / **Control + C** (macOS, Linux).

Установка и удаление пакетов

Пакет (зависимость) — готовая библиотека или утилита от других разработчиков. npm состоит из трёх частей:

- **Реестр** (registry) — общая база всех пакетов.
- **Сайт npmjs.com** — поиск пакетов и их документация (там же инструкция по установке).
- **CLI** (Command-Line Interface — интерфейс командной строки) — команды npm в терминале.

Действие	Команда	Короткая форма
Установить все зависимости проекта	npm install	npm i
Установить пакет	npm install validator	npm i validator
Удалить пакет	npm uninstall validator	npm r validator

После установки пакет появляется в `node_modules` и в поле `dependencies` файла `package.json`; после удаления — исчезает оттуда.

Пример: библиотека validator

Библиотека для проверки строк (например, ввода в форму). После `npm i validator` импортируем её в `main.js`:

```
import validator from "validator";

console.log(validator.isEmail("mango@mail.com")); // true
console.log(validator.isEmail("Mangodogmail.com")); // false
```

Результат импорта — объект с методами. Проверяем в консоли браузера после `npm run dev`.

Типы зависимостей

Аналогия с тортом: продукты входят в сам торт, а миксер и миска нужны только при готовке. Так же и пакеты:

Флаг	Поле в package.json	Для чего	Пример
--save (по умолчанию)	dependencies	Входит в финальный продукт	validator
--save-dev	devDependencies	Нужен только при разработке	vite

```
npm i validator          # = --save → dependencies
npm i vite --save-dev    # → devDependencies
npm uninstall vite --save-dev
```

При удалении пакета указывай **тот же флаг**, что и при установке.

Модульность кода

Зачем нужны модули

Большой проект разбивают на файлы, каждый из которых отвечает за свою часть функциональности. Это даёт три главных плюса:

Плюс	Суть
Поддержка	Модуль самодостаточен: его можно менять и расширять, почти не рискуя сломать остальной код
Своя область видимости	Переменные модуля не попадают в глобальную область, поэтому имена не конфликтуют («не засоряют» глобальное пространство)
Повторное использование	Готовый модуль подключается в любой проект без копирования и переделки кода

ES-модули (ESM)

ESM (ECMAScript Modules) — стандартная система модулей JavaScript. **Модуль** — это обычный JS-файл: он **экспортирует** (открывает наружу) нужные значения, а другие файлы их **импортируют**.

Современные браузеры поддерживают ESM сами, но в проектах модули обычно собирает сборщик — например, Vite, где всё работает без дополнительной настройки.

Экспорт и импорт по умолчанию (default)

В модуле может быть **только один** экспорт по умолчанию — «главное» значение модуля (основная функция или объект).

```
// makeMessage.js
const makeMessage = username => {
  return `Welcome, ${username}!`;
};

export default makeMessage;

// index.js
import makeMessage from "./makeMessage";

console.log(makeMessage("Jacob")); // "Welcome, Jacob!"
```

- `./makeMessage` — относительный путь к файлу модуля (строкой).
- Имя при импорте default-значения **можно выбрать любое**.

Именованный экспорт и импорт (named)

Экспортирует значения под конкретными именами. Таких экспортов в модуле может быть **сколько угодно**.

```
// makeMessage.js
export const makeMessage = username => {
  return `Welcome, ${username}!`;
};

export const levels = ["easy", "medium", "hard"];

// index.js
import { makeMessage, levels } from "./makeMessage";

console.log(makeMessage("Jacob")); // "Welcome, Jacob!"
console.log(levels);                // ["easy", "medium", "hard"]
```

- Имена в фигурных скобках **должны совпадать** с именами экспорта.

- Именованный экспорт импортируется только именованным импортом (в `{ }`), `default` — только без скобок.

Сравнение

	Default	Named
Экспорт	<code>export default value</code>	<code>export const name = ...</code>
Импорт	<code>import anyName from "..."</code>	<code>import { name } from "..."</code>
Сколько в модуле	Один	Сколько угодно
Имя при импорте	Любое	Как при экспорте (или через <code>as</code>)

Переименование при импорте (`as`)

Ключевое слово `as` задаёт своё имя именованному импорту. Нужно, когда имя слишком длинное, слишком общее или уже занято в текущем файле.

```
// myModule.js
export const original = "Hello, World!";

// app.js
import { original as renamed } from "./myModule.js";

console.log(renamed); // "Hello, World!"
```

Импорт пространства имён (`* as`)

Импортирует **все именованные экспорты** модуля одним объектом — они становятся его свойствами. Удобно, когда экспортов много, и помогает избежать конфликтов имён.

```
// user.js
export const name = "Mango";
export const age = 26;
export const email = "mango@mail.com";

// app.js
import * as user from "./user.js";

console.log(user.name); // "Mango"
console.log(user.age); // 26
console.log(user.email); // "mango@mail.com"
```